

CS285 LECTURE 5 • ZERO-TO-HERO FIELD MANUAL

Policy Gradients: Trial-and-Error Learning

A Comprehensive, Intuitive Textbook: Score Functions, Continuous Gaussian Control, Baselines, and Multi-Domain Architectures

Instructor: Prof. Sergey Levine (UC Berkeley RAIL Lab) **Target Audience:** Complete Beginners to Advanced Robotics Practitioners

1. Why Policy Gradients? (The Non-Differentiable World)

In traditional deep learning (like computer vision), we compute gradients by differentiating through mathematical equations using the chain rule. If an image is slightly too dark, backpropagation computes $\frac{\partial \mathcal{L}}{\partial W}$ and updates the convolution weights.

⚠ THE PHYSICS CALCULUS BREAKDOWN

In robotics, your neural network commands a physical knife to touch a ripe tomato. **A tomato skin tearing under a blade is a discontinuous physical fracture.** Contact friction suddenly shifts between stick and slip regimes. Viscoplastic pulp flows irreversibly.

If you attempt to write down the calculus derivative $\frac{\partial \text{FruitDamage}}{\partial \text{MotorVoltage}}$, the derivative is either **zero** (flat plateaus where nothing happens) or **undefined infinity** (the microsecond the skin ruptures). Standard analytical backpropagation through physical contact is mathematically dead!

1.1 The Breakthrough: Optimizing the Expected Return

Instead of trying to differentiate through complex physics equations, **Policy Gradients** bypass physics entirely. We treat the physical environment as a black box and directly optimize the *Expected Total Reward*:

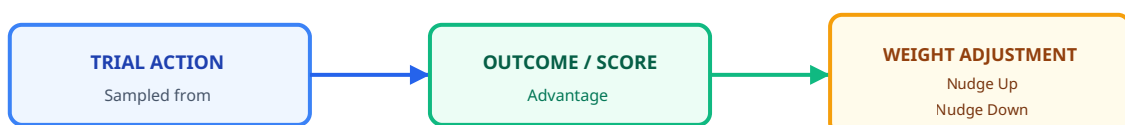
$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)] = \int \mathbb{P}(\tau \mid \theta) R(\tau) d\tau$$

🐶 THE PUPPY TRAINING ANALOGY ("GOOD BOY!" THEOREM)

When you teach a puppy to sit on command:

- You cannot open the puppy's skull with a wrench and rewire its brain synapses directly.
- You say "Sit!". The puppy tries random actions: it barks, rolls over, chases its tail, and wiggles.
- Eventually, by pure chance, the puppy's hind legs sit on the floor.
- **Instant Reinforcement:** You shout "GOOD BOY!" and give it a piece of steak (Reward).
- The puppy's brain automatically strengthens the connection: "*Whenever I hear that command, sitting leads to steak!*"

The Policy Gradient Theorem is the exact mathematical formulation of "Good Boy!" We let the robot execute trial actions. When an action produces positive reward, we nudge the neural network's weights to make that action more probable. When it produces a penalty, we push the weights in the opposite direction!



2. The Policy Gradient Theorem (Exhaustive Parameter Anatomy)

Here is the foundational mathematical equation derived by Ronald Williams in 1992 (the REINFORCE algorithm):

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot \hat{Q}_t \right]$$

Symbol	Formal Name	Plain English Meaning	Physical Effect in Robotics
$\nabla_{\theta} J(\theta)$	Policy Gradient Vector	The compass direction in neural network weight space that maximizes total score.	Updates policy weights: $\theta \leftarrow \theta + \alpha \nabla_{\theta} J$.
$\pi_{\theta}(a_t s_t)$	Action Likelihood	Probability of picking action a_t in state s_t .	Output of the robot's policy network.
$\nabla_{\theta} \log \pi_{\theta}$	Score Function	The Steering Wheel: "Which direction in weight space makes action a_t more probable?"	Gives the sensitivity of action likelihood to network parameters.
$\hat{Q}_t$	Performance Multiplier	The Gas Pedal & Reverse Gear: "How good or bad was this action?"	If $\hat{Q} > 0$, press gas pedal. If $\hat{Q} < 0$, put into reverse!

PLAIN ENGLISH TRANSLATION OF THE POLICY GRADIENT EQUATION

"For every action the robot executed, calculate the direction in neural network weights that would make that action **MORE** likely. Then multiply that direction by the action's score ($\hat{Q}$). If the action was great, nudge the weights to do it more often. If the action caused a crush penalty, nudge the weights in the exact opposite direction!"

3. Four Real-World Case Studies for Policy Gradients

Let's observe how Policy Gradients optimize decisions across four completely different engineering systems:

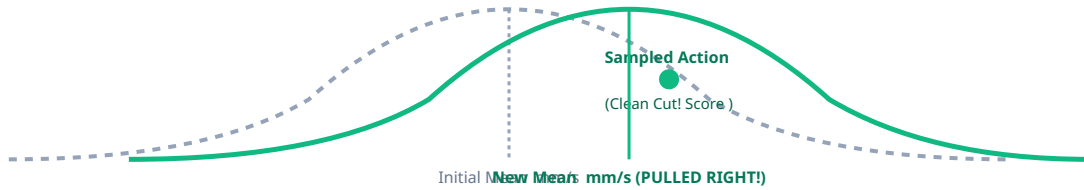
Domain	What is the Action (a_t)?	What is the Score Multiplier ($\hat{Q}$)?	What the Policy Gradient Does
1. Soft Fruit Slicing (Our Lab's Research)	Continuous downward feed delta Δv_z and lateral sawing speed v_{slice} .	Penetration progress (+5) minus skin crushing penalty ($-4 \times \text{ExcessForce}^2$).	Pulls the mean sawing speed upward while softening vertical feed rate upon skin contact.
2. Autonomous Car (Highway Driving)	Steering wheel continuous angle $\delta \in [-30^\circ, +30^\circ]$ and brake pressure.	Velocity tracking bonus (+2) minus lane departure penalty (-50) and jerk penalty.	Reinforces gentle, anticipatory steering nudges while suppressing sharp swerving.
3. ChatGPT Alignment (RLHF / PPO)	Discrete token ID chosen from 50,000 vocabulary words at position t .	Human feedback reward model score (+8.5) minus KL divergence penalty.	Increases the probability of polite, helpful tokens and suppresses hallucinations.
4. Quadruped Robot (Locomotion)	12 joint motor target angles commanded to low-level PD controllers.	Forward velocity reward (+3) minus foot slip penalty and motor torque heating.	Nudges joint trajectories to synchronize into an energy-efficient trotting gait.

4. Continuous Gaussian Policies: Controlling Real Robot Actuators

In discrete games (like Pong), the network outputs a softmax over 2 buttons (Up/Down). In your dual-arm setup, the robot commands continuous real-valued numbers (e.g. feed rate $v_z = 2.45$ mm/s, stiffness $\Delta K = 350$ N/m).

How can a neural network output continuous numbers while still exploring? It outputs the parameters of a **Gaussian (Normal) Distribution**:

$$\pi_{\theta}(a | s) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(a - \mu_{\theta}(s))^2}{2\sigma^2}\right)$$



4.1 Analytical Gaussian Score Function

When we differentiate the Gaussian log-probability with respect to the mean μ , we obtain an extraordinarily elegant result:

$$\nabla_{\mu} \log \pi_{\theta}(a | s) = \frac{a - \mu_{\theta}(s)}{\sigma^2}$$

Term	Plain English Meaning	Tomato Slicing Value	Physical Role
$\mu_{\theta}(s)$	The Robot's Intended Action (Mean)	$\mu = 2.0$ mm/s feed rate	What the robot thinks is best.
σ	Exploration Noise (Standard Dev)	$\sigma = 0.5$ mm/s wiggle room	How widely the robot experiments.
$a - \mu$	Exploration Deviation	$2.8 - 2.0 = +0.8$ mm/s	Did the robot push faster (> 0) or slower (< 0)?
$\frac{a - \mu}{\sigma^2} \cdot \hat{Q}$	Gradient Pull Force	$\frac{+0.8}{0.25} \times (+12) = +38.4$	Pulls the intended mean μ towards successful cuts!

5. The Baseline: Why We Must Grade on a Curve

🎓 THE COLLEGE EXAM ANALOGY (WHY RAW SCORES DECEIVE)

Imagine an entire class takes an exam so easy that every student scores between 90 and 100 points. If you score 91, did you do well? **No, you were in the bottom 5% of the class!**

In RL, if all rewards are positive (e.g. scores are +100, +105, +95):

- Raw policy gradients will **push UP every action**, even the terrible ones that only scored +95!
- The network becomes confused because it is being told to reinforce everything.

The Fix (The Baseline): We subtract the average expected score $b(s) = V(s)$:

$$\hat{A}_t = Q(s_t, a_t) - V(s_t)$$

Now, an action that scores +95 when the average was +100 gets an advantage of -5.0 ! It gets suppressed, while an action that scores +105 gets +5.0 and gets reinforced. **Subtracting a baseline is mathematically 100% unbiased (by the EGLP Lemma), but reduces gradient variance by 90%!**

6. Concrete Numerical Walkthrough: Updating a Policy with Numbers

Let's walk through an exact numerical calculation of one policy gradient step:

Step	Variable / Expression	Numerical Value	Physical Meaning
1	Current Mean Policy $\mu_\theta(s)$	2.0 mm/s	Robot intends to push blade at 2.0 mm/s.
2	Exploration Noise σ	0.5 mm/s	Variance $\sigma^2 = 0.25$.
3	Sampled Action a	2.6 mm/s	Random sample with noise $\epsilon = +1.2$.
4	Observed Return G	+18.0	Blade cleanly punctured skin into pulp!
5	Baseline Prediction $b(s) = V(s)$	+12.0	Critic expected average performance to be +12.0.
6	Advantage Multiplier $\hat{A}$	$18.0 - 12.0 = +6.0$	Action beat the curve by +6.0 points!
7	Gaussian Score $\frac{a-\mu}{\sigma^2}$	$\frac{2.6-2.0}{0.25} = +2.4$	Compass direction pointing toward faster feeds.
8	Gradient Step ($\alpha = 0.01$)	$\Delta\mu = 0.01 \times (2.4) \times (6.0) = +0.144$	Mean feed rate increases from 2.0 $\rightarrow$ 2.144 mm/s!

7. "Diary of a Training Run" for Policy Gradients

Iteration Phase	Policy Behavioral Evolution	TensorBoard Diagnostics
Phase 1: High Variance (Iters 0 – 50)	Gradients swing wildly in direction. Mean feed rate oscillates violently between -10 and $+10$ mm/s.	Policy loss fluctuates wildly. Standard deviation σ remains wide ($0.8 - 1.0$).
Phase 2: Baseline Lock-in (Iters 50 – 200)	The Critic learns accurate baseline values $V(s)$. Advantages become clean and zero-centered. Policy stabilizes.	Gradient norm drops by 70%. Mean feed rate settles into positive cutting territory (≈ 2.5 mm/s).
Phase 3: Standard Dev Decay (Iters 200 – 1000)	Robot becomes confident in its cutting trajectory. Exploration noise σ gently decays from $0.5 \rightarrow 0.15$.	Entropy drops steadily toward optimal deterministic control. Cut success rate reaches 98%.

8. Production PyTorch Policy Gradient Implementation

COMPLETE PYTORCH REINFORCE WITH CONTINUOUS GAUSSIAN POLICY & BASELINE

```
import torch
import torch.nn as nn
from torch.distributions import Normal

class GaussianPolicy(nn.Module):
    def __init__(self, state_dim=33, action_dim=6):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 256), nn.Tanh(),
            nn.Linear(256, 256), nn.Tanh()
        )
        self.mean_head = nn.Linear(256, action_dim)
        # Learnable log-standard deviation parameter
        self.log_std = nn.Parameter(torch.zeros(action_dim))

    def forward(self, state):
        features = self.net(state)
        mean = self.mean_head(features)
        # Clamp log_std to prevent numerical explosion or standard dev collapse
        std = torch.exp(torch.clamp(self.log_std, min=-2.0, max=1.0))
        return Normal(mean, std)

def compute_policy_loss(dist, actions, advantages, c_entropy=0.01):
    # 1. Compute log-probability of taken actions: log pi(a|s)
    log_probs = dist.log_prob(actions).sum(dim=-1)

    # 2. Policy Gradient Objective: -E[ log pi(a|s) * Advantage ]
    policy_loss = -(log_probs * advantages.detach()).mean()

    # 3. Entropy Regularization: Prevents premature exploration collapse
    entropy_loss = -c_entropy * dist.entropy().sum(dim=-1).mean()

    return policy_loss + entropy_loss
```

9. Practitioner's Failure Modes & Debugging Checklist

Failure Mode	The Hidden Cause	How to Fix It Immediately
1. Standard Deviation Collapse	The network figures out that setting $\sigma \rightarrow 0$ eliminates penalty risk. The policy freezes into a deterministic rut and stops exploring completely.	Clamp $\log \sigma$ with a minimum floor: <code>torch.clamp(log_std, min=-2.0)</code> and add an Entropy Bonus (<code>c_entropy=0.01</code>).
2. Forgetting to Detach Advantages	If you don't call <code>advantages.detach()</code> , gradients flow backward through the Critic network into the policy loss, corrupting Critic weights.	Always call <code>advantages.detach()</code> before multiplying by <code>log_probs</code> .
3. Small Batch Size Noise	Policy gradients on small batches (e.g. 32 steps) are mostly random noise. The policy takes erratic steps and diverges.	Collect at least 2,048 to 4,096 parallel environment steps before each policy gradient update.
4. Exploding Learning Rates	Setting learning rate $\alpha = 1 \times 10^{-2}$ causes policy weights to jump into saturated Tanh regions where all gradients vanish.	Use standard Adam step size: $\alpha = 3 \times 10^{-4}$.

CS285 Lecture 5 Zero-to-Hero Guide • DEX-ROB Lab (Tianjin University) • Prof. Shan An